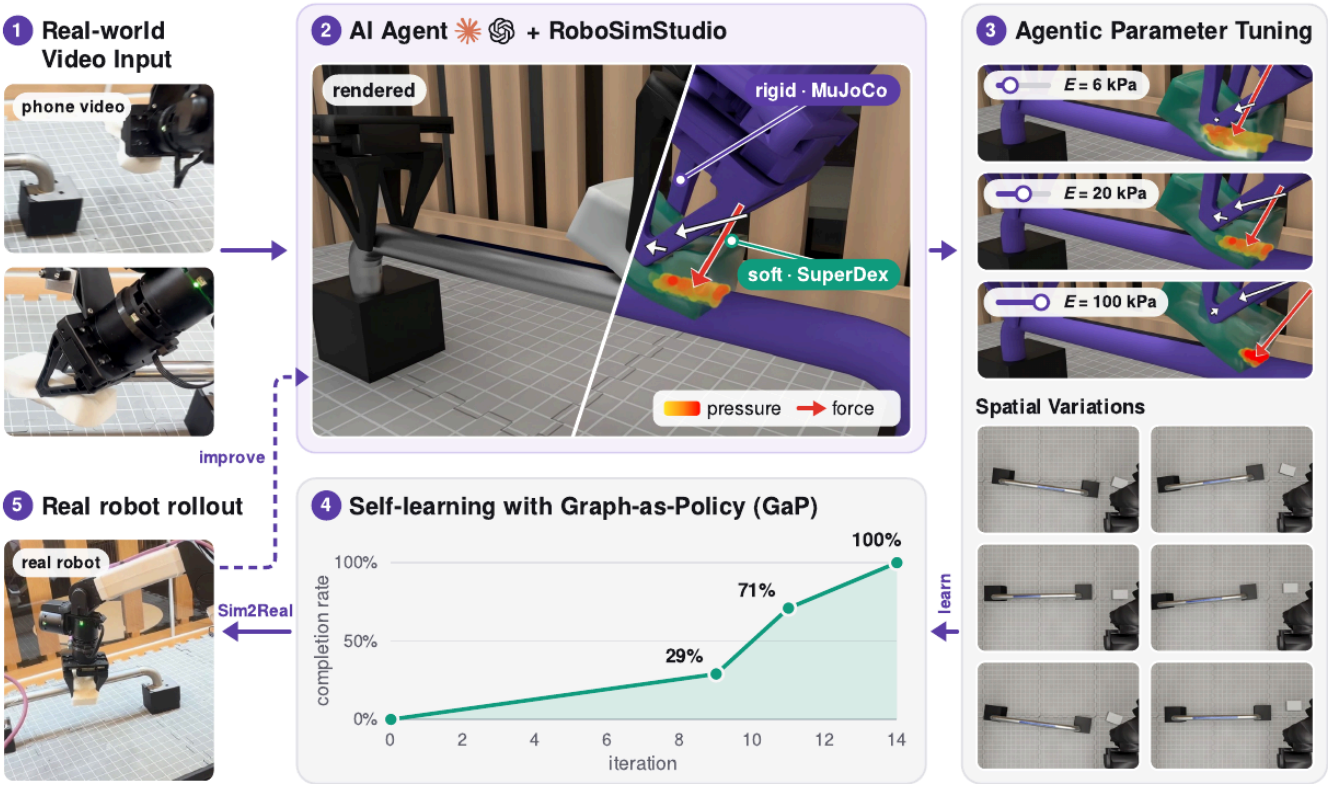
RoboSimStudio: 분석 리포트

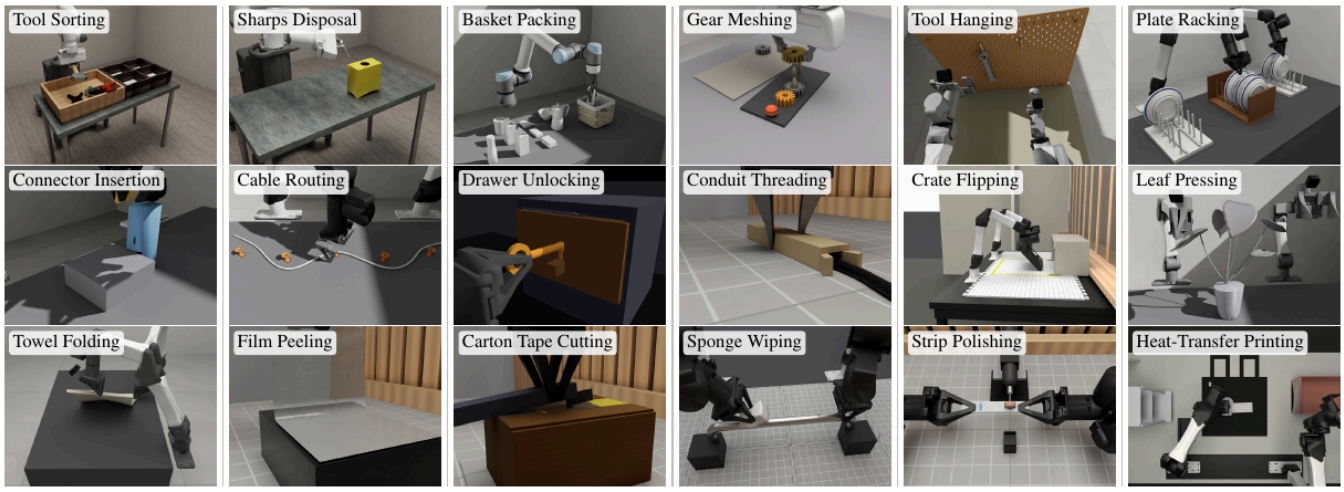
게시용 버전 · 원문 https://arxiv.org/abs/2610.18835 · arXiv:2610.18835v2 · UC Berkeley, NVIDIA (Chen, Xie, Yoo, Zhu, Magner, Zhu, Kim, Heiden, Coumans, Macklin, Huang, Goldberg) · 코드 robosimstudio/robosimstudio @ 81f57c9 (Apache-2.0) · 분석일 2026-10-29 ·
논문 그림 © 저자, CC BY 4.0



논문 Fig. 1: 스마트폰 영상 → 코딩 에이전트 + RoboSimStudio로 다중 물리 시뮬레이션 → 물성·배치 변형 → GaP 정책 개선 → 실제 로봇.

핵심 요약

1. **무엇인가:** Newton 위에 solver·접촉 모델·물성을 편집 가능한 항목으로 드러낸 Real2Sim 도구다. 사람은 브라우저 studio와 힘 피드백 리더 암으로, 코딩 에이전트는 저장하면 다시 빌드되는 live 페이지와 CLI로 같은 장면을 고친다.
2. **논문보다 저장소가 실체다:** 정량 결과가 거의 없고, Table 1은 RSS가 10/10인 기능 목록이다. 반면 공식 저장소(Apache-2.0)는 설치 문서, 예제, 테스트, CPU 경로까지 갖춘 쓸 수 있는 도구다.
3. **핵심 자산은 비공개다:** Fig. 5의 18개 과제 정의, GaP 닦기 정책과 30개 배치, UR5e, 실제 로봇 코드가 공개 저장소에 없다. 논문 결과는 재현할 수 없다.
4. **공개 twin은 물리적 잡기를 우회한다:** 마찰만으로는 스펀지가 69-226 mm 미끄러져 닦인 비율이 0.72에 그친다. 그래서 폼 노드 106개를 손에 고정(latch)해 0.96을 얻는다. 영상 속 20 kPa 스펀지 대신 80 kPa 폼도 쓴다. 같은 저장소의 에이전트 규칙은 latch를 금지한다.
5. **영상만으로는 물성이 정해지지 않는다:** toy로 계산하면 1080p 영상에서 E > 100 kPa는 사실상 식별 불가하고, 80 vs 100 kPa는 0.36σ 차이다. 힘 관측은 오차가 일정하다(0.4 kPa). 강성을 "맞추기"보다 "변형"한 선택과 힘 피드백의 동기를 뒷받침한다.
6. **GaP 곡선의 29%·71%는 30개 배치의 성공률로 나올 수 없는 값이다.** 닦인 면적 같은 연속량으로 읽어야 한다.
7. **구현 가능성:** Linux x86-64 + Python 3.12가 기준이다. 이 맥에서는 CPU 경로로 강체 장면과 에이전트 루프를 시험할 수 있어 보이지만, 설치 승인이 필요하다. 변형체 대부분, 경로 추적 렌더러, GaP·cuRobo는 NVIDIA GPU가 필요하다.



논문 Fig. 5: RoboSimStudio로 만든 contact-rich 과제 18개. 공개 저장소에는 이 중 3개(스펀지 닦기, 케이블 배선, 수건 접기)의 데모만 있다.

01. 논문 분석: RoboSimStudio (arXiv:2610.18835)

- 제목: *RoboSimStudio: An Interactive Toolkit for Human and Agent Rapid Real2Sim of Robot Manipulation Tasks* (PDF 표지 제목은 "An Interactive Toolkit for Rapid Real2Sim of Robot Manipulation Tasks")
- 저자: Kaiyuan Chen*, Shuangyu Xie*, Uksang Yoo, Xinyue Zhu, Cael Magner, Alvin Zhu, Daehwa Kim, Eric Heiden, Erwin Coumans, Miles Macklin, Spencer Huang, Ken Goldberg (UC Berkeley, NVIDIA)
- 공개: arXiv v1 2026-10-13, **v2 2026-10-14** (14쪽) · 코드 https://github.com/robosimstudio/robosimstudio (Apache-2.0, 확인 커밋 `81f57c9` 2026-10-28, 논문 시점 첫 커밋 `73739ab` 2026-10-14) · 프로젝트 https://robosimstudio.github.io
- 라이선스: 논문 **CC BY 4.0** (arXiv abs 페이지 확인)
- 분석 범위: 본문 9쪽과 참고문헌(v2), v1과의 텍스트 대조, 프로젝트 페이지, 공식 저장소 전체(코드·예제·설치 문서·테스트)

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: 시뮬레이터의 **solver, 접촉 모델, 물성 값을 명시적으로 드러내고 바로 고칠 수 있게** 만든 Newton 기반 도구입니다. 사람(브라우저 화면, 슬라이더, 힘 피드백 리더 암)과 코딩 에이전트(Claude Code, Codex)가 같은 장면을 고쳐 가며, 실제 영상과 비슷하게 움직이는 다중 물리 시뮬레이션(Real2Sim)을 빠르게 만들게 하려는 것입니다.

기여는 세 가지입니다(Sec. 1).

1. 접촉 모델과 물성을 바꿀 수 있는 contact-rich 조작용 시뮬레이션 도구 RoboSimStudio(RSS)
2. 사람과 코딩 에이전트가 함께 쓰는 Real2Sim 인터페이스
3. 시각·힘 피드백으로 만든 조작 과제 18개, 그리고 스마트폰 영상에서 코딩 에이전트가 다중 물리 시뮬레이션을 만든 Real2Sim2Real 파일럿 1건

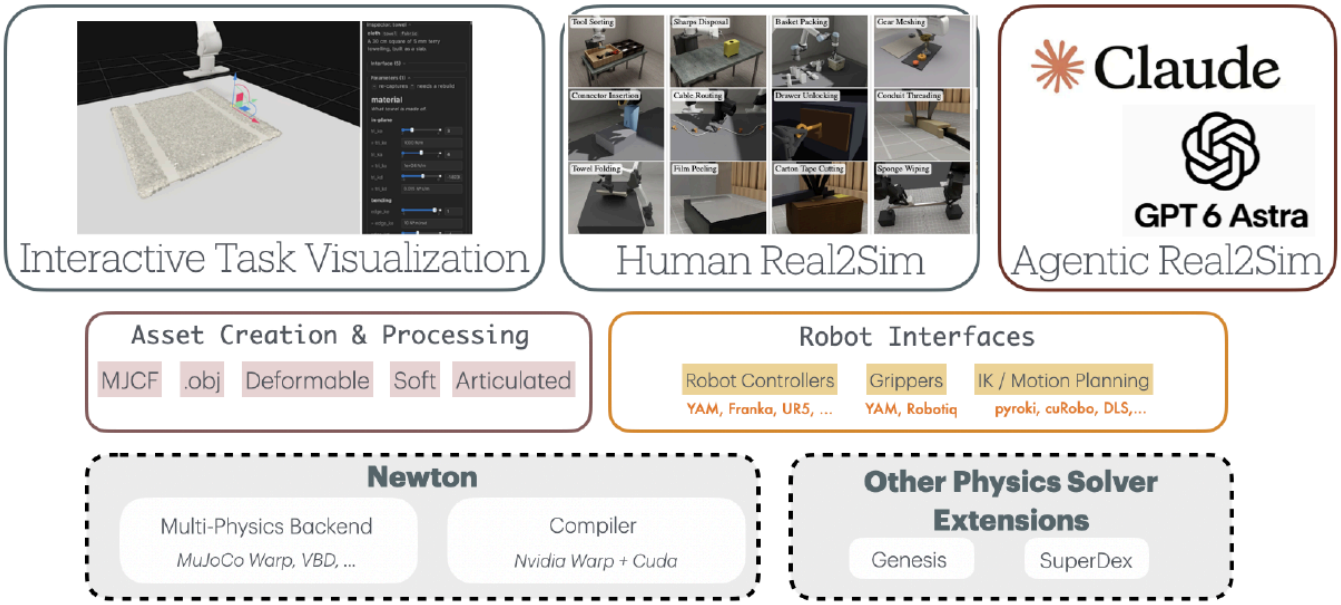
논문은 스스로를 "개발 중인 시스템과 파일럿 연구"라고 부릅니다(Sec. 5). 정량 실험은 거의 없고, 대부분의 결과는 정성적 시연입니다.

---

2. Method

2.1 구조 (Fig. 2, Sec. 3)

```
영상·사진 (사람·로봇 시연, 스마트폰)
        │  사람: 브라우저 studio (Viser) — 선택·드래그·인스펙터 슬라이더
        │  코딩 에이전트: scene.py를 고치고 실행·관찰하는 반복 루프
        ▼
장면 = 로봇 + 물체 + 물리 배치(arrangement)
   물체는 "무엇인지"(강체, 관절, 천, 케이블, 스펀지 …)만 선언
   장면이 "어느 solver가 어떤 물리를 맡는지" 선언 → 바꿔도 과제 코드는 그대로
        ▼
Newton (Warp)  — 관절·강체: MuJoCo-Warp   — 천·케이블: VBD
        └ 외부 엔진 co-sim: SuperDex(FEM 스펀지), Genesis   ← 접촉력·자세·wrench 교환
        ▼
변형(물성 E, 공간 배치) → Graph-as-Policy(GaP)로 정책 개선 → 실제 로봇
```



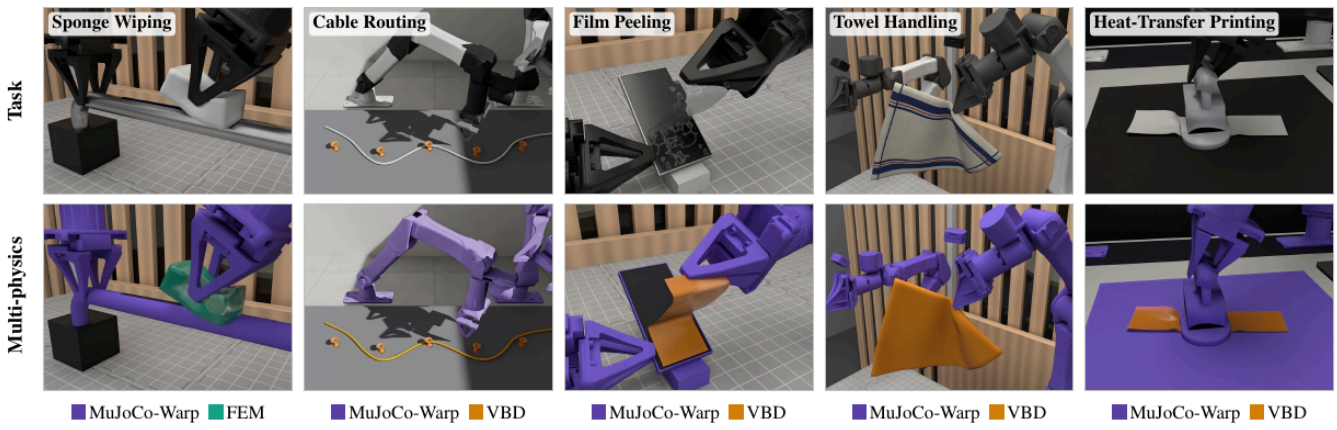
논문 Fig. 2: 자산 처리, 로봇 인터페이스, Newton 다중 물리 백엔드와 외부 solver(Genesis, SuperDex).

2.2 로봇 인터페이스

Franka Panda, UR5e, 단팔·양팔 YAM의 공통 인터페이스가 있습니다(Sec. 3). IK는 PyRoKi 같은 solver를 바꿔 끼울 수 있고, 계획은 cuRobo를 씁니다. 정책은 관절 공간이나 엔드이펙터 공간으로 명령합니다. **조작 물체의 privileged 상태는 로봇이 볼 수 있는 관측과 분리**합니다.

2.3 다중 물리와 결합

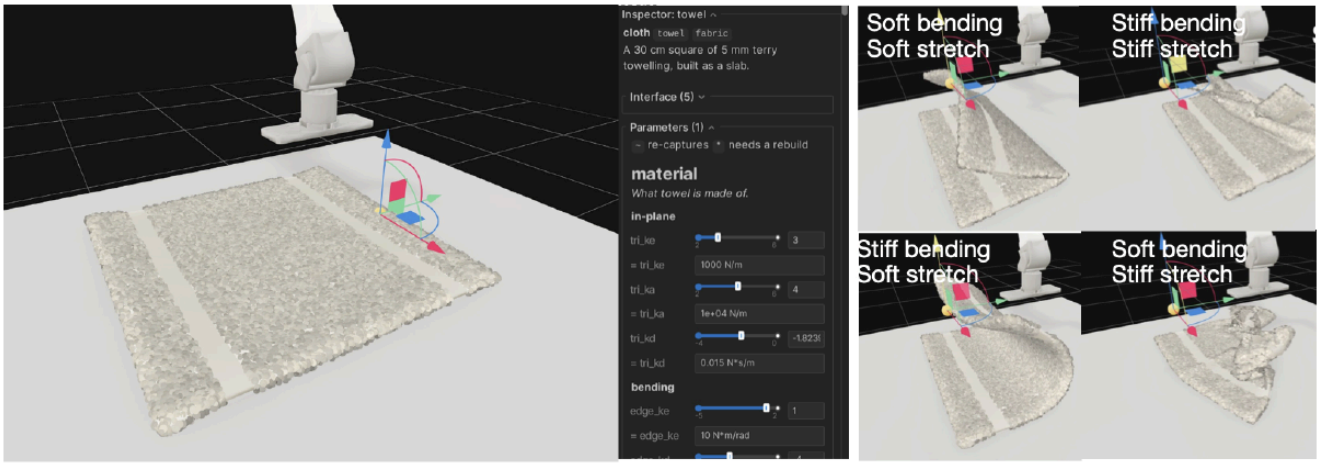
기본값은 다음과 같습니다. 관절 로봇은 MuJoCo-Warp, 케이블과 천은 Vertex Block Descent(VBD), 연성체는 외부 엔진(SuperDex, Genesis)과 co-sim입니다. Newton 내부 solver끼리는 접촉력을, 외부 엔진과는 자세와 접촉 wrench를 주고받아 결합합니다. solver, 결합 방식, 접촉 모델, 재료 값은 장면의 명시적 항목이라 과제를 다시 쓰지 않고 바꿀 수 있습니다(Fig. 3).



논문 Fig. 3: 다섯 과제의 solver 배치. MuJoCo-Warp(보라), FEM(청록), VBD(주황).

2.4 사람용 인터페이스 (Fig. 4)

Viser 기반 브라우저 studio입니다. 물체를 선택·드래그·검사하고 로봇 팔을 직접 움직일 수 있습니다. 인스펙터는 물성을 **단위·범위와 함께** 보여 주고, 실행 중 바꿀 수 있는 값과 장면을 다시 만들어야 하는 값을 구분해 표시합니다.



논문 Fig. 4: 수건 인스펙터와 휨·늘어남 강성 네 조합의 같은 접기.

2.5 에이전트용 인터페이스

코딩 에이전트는 사람과 같은 루프를 돕니다. 장면을 고치고, 행동을 실행하고, 결과를 관찰합니다. RSS는 rollout을 로봇 동작 primitive로 나누고, 시뮬레이터 상태를 **시공간 텍스트 피드백**으로 바꿔 줍니다. 피드백에는 목표 오차, 물체 움직임, 접촉 사건, 잡기·미끄러짐, 과제 진행도가 들어갑니다. 사람은 에이전트가 작업하는 동안 비동기로 같은 화면을 보며 값을 고치거나 지시할 수 있습니다(human-in-the-loop).

---

3. 사례 연구 (Sec. 4)

3.1 시각 피드백으로 하는 사람 Real2Sim (Sec. 4.1, Fig. 5)

사용자가 자산을 고르고 배치하고 물리 모델을 고른 뒤, 실제 거동과 시뮬레이션이 **정성적으로** 맞을 때까지 접촉·재료 값을 조정했습니다. 이렇게 과제 18개를 만들었습니다. 예로 든 것은 세 가지입니다.

- **커넥터 삽입:** USB-C를 1 mm 미만 여유로 꽂으려고 메쉬 기반 접촉과 충돌 해상도를 조정했습니다.
- **잎 누르기:** 쉘 시뮬레이션으로 얇은 잎의 휨과 회복을 표현했습니다.
- **띠 연마:** 접촉력에서 Preston 식으로 재료 제거량을 계산했습니다.

본문은 한 문장 안에서 "세 과제"와 "다섯 예"를 함께 쓰고, 실제로는 셋만 나열합니다. 편집 흔적으로 보입니다.

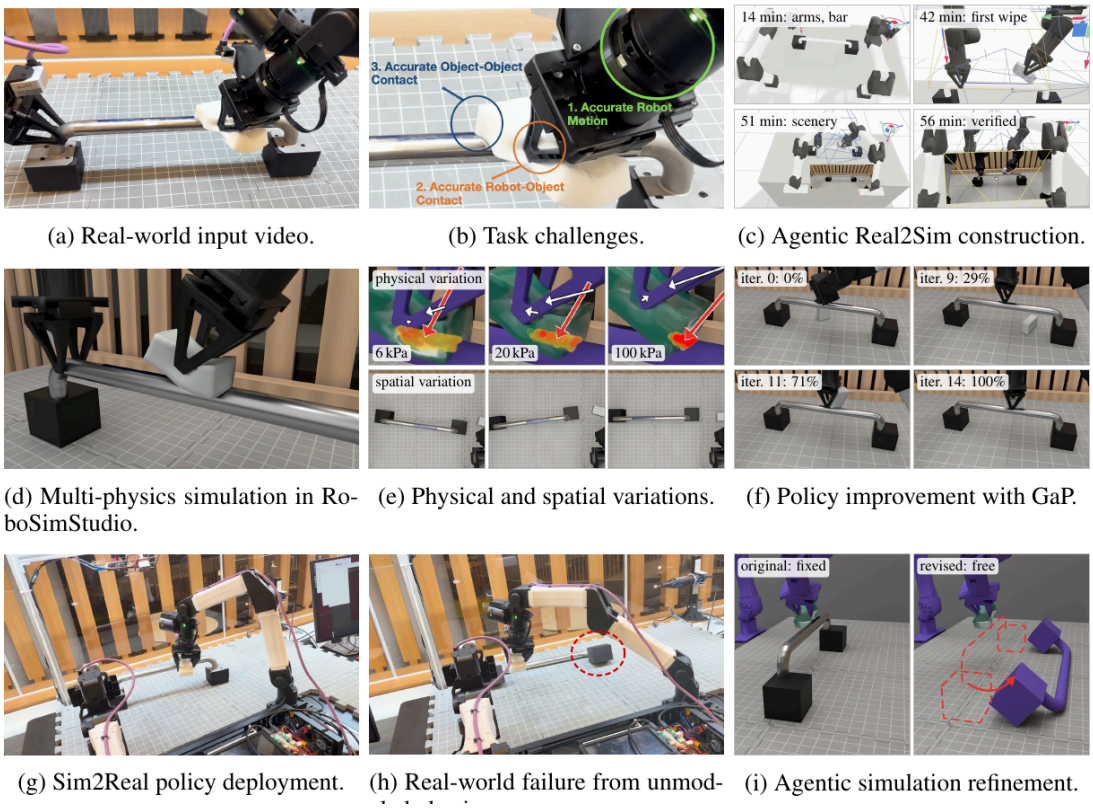
3.2 힘 피드백으로 하는 사람 Real2Sim (Sec. 4.2, Fig. 6)

저가 리더 암(논문은 GELLO[58] 인용)으로 시뮬레이션 로봇을 조종하고, 시뮬레이션 접촉을 리더 모터 토크로 되돌려 줍니다. 명령 자세와 시뮬레이션 자세의 차이가 피드백 신호입니다. 시뮬레이션 지연을 접촉으로 착각하지 않도록 속도 의존 deadband를 두고, 시뮬레이션이 멈추거나 뒤처지면 피드백을 끕니다. 그리퍼는 명령값이 아니라 측정된 열림을 씁니다. 공구 걸기 과제를 시연했지만 정량 결과는 없습니다.

3.3 에이전트 Real2Sim2Real (Sec. 4.3, Fig. 7)

양팔 스펀지 닦기 과제입니다. 한 팔이 막대를 잡고 다른 팔이 폼 스펀지로 마커 줄을 지웁니다.

단계	보고 내용
구축	CAD도 보정 카메라도 없는 스마트폰 영상 하나로 Claude Fable 5.1(Claude Code)과 GPT-6 Astra(Codex)가 각자 "약 1시간"에 다중 물리 시뮬레이션을 만듦. 한 에이전트는 56분에 검증 완료(14분 팔·막대, 42분 첫 닦기, 51분 배경, 56분 검증)
물리	MuJoCo-Warp(Newton) + SuperDex(스펀지 FEM). 접촉력과 압력 분포 확인
변형	스펀지 강성 6·20·100 kPa, 막대·스펀지 배치 30개
정책	GaP가 14번 반복으로 시뮬레이션에서 닦인 면적을 0% → 100%로 올림(반복 9: 29%, 11: 71%)
실제	개선된 정책을 실제 로봇에서 실행해 닦기 완료. 한 rollout에서는 시뮬레이션에서 고정했던 막대 받침이 밀려나 실패
정제	실패 영상과 설명을 받은 에이전트가 고정 제약을 자유 강체로 바꿈 → 수정된 시뮬레이션은 막대가 밀리고 기우는 현상을 재현



(a) Real-world input video. (b) Task challenges. (c) Agentic Real2Sim construction.

(d) Multi-physics simulation in RoboSimStudio. (e) Physical and spatial variations. (f) Policy improvement with GaP.

(g) Sim2Real policy deployment. (h) Real-world failure from unmod- (i) Agentic simulation refinement.

논문 Fig. 7: 스펀지 닦기 Real2Sim2Real. (c) 에이전트 구축, (e) 강성·배치 변형, (f) GaP 개선, (h) 실제 실패, (i) 시뮬레이션 수정.

4. 해석

- **이 논문의 실체는 도구입니다.** 성능 표가 없고, Table 1은 다른 프레임워크와의 기능 비교입니다. 그런데 이 표에서 RSS는 10개 행이 모두 ✓이고, RSS만 ✓인 5행은 전부 이 논문이 정의한 워크플로입니다(`04` C1). 비교라기보다 기능 목록에 가깝습니다.
- **GaP 곡선의 단위가 그림마다 다릅니다.** Fig. 1은 y축을 "completion rate"라고 쓰고, 본문과 Fig. 7f는 "simulated wiped area"라고 씁니다. 29%와 71%는 30개 배치의 성공 비율로는 나올 수 없는 값입니다(k/30 = 26.7, 30.0, 70.0, 73.3%). 따라서 닦인 면적 같은 연속량이거나 다른 분모입니다(`04` C2).

- **실제 로봇 결과는 시행 횟수가 없습니다.** "실제 로봇에서 닦기를 완료했다"와 "한 rollout이 실패했다"만 있습니다. 시뮬레이션-실제 일치도는 "주로 정성적 관찰"이라고 저자들도 씁니다(Sec. 5).
- **공식 저장소가 논문보다 훨씬 구체적입니다.** 다음 사실은 저장소에서만 드러납니다.
  - 공개 twin의 스펀지는 friction만으로 잡히지 않습니다. 그래서 패드 주변 폼 노드 106개를 손에 고정(latch)하고, 영상의 부엌 스펀지(약 20 kPa) 대신 80 kPa 폼을 씁니다(`04` D3·D4).
  - 18개 과제 대부분과 UR5e, GaP 닦기 정책, 30개 배치는 공개 저장소에 없습니다(`04` D1·D2·D6).

---

5. Limitations

5.1 저자가 밝힌 한계 (Sec. 5)

- 개발 중인 시스템이고 파일럿 연구입니다.
- 에이전트의 시뮬레이션 구축은 RSS가 제공하는 재료 모델, 물리 결합, 로봇 자산에 의존합니다.
- 외부 엔진 co-sim은 계산 부담이 있습니다.
- sim-to-real은 과제 하나로만 평가했고, 일치도는 주로 정성적입니다.
- 향후 과제로 시뮬레이션 정확도, 접촉력, 물체 궤적, 구축 시간, 실제 성공률의 체계적 정량 평가를 들었습니다.

5.2 분석하면서 추가로 발견한 한계와 의문점

1. **"영상과 맞을 때까지"의 판정 기준이 없습니다.** 맞았다는 판단이 눈으로 하는 정성 비교입니다. 영상만으로는 강성 같은 값이 잘 정해지지 않습니다. toy로 계산하면 1080p 영상에서 판독 잡음이 0.5 px만 있어도, E가 약 100 kPa를 넘으면 E 추정의 상대 표준오차가 50%를 넘습니다. 80 kPa와 100 kPa는 0.4σ 차이라 구분되지 않습니다(`03` §2). 논문이 강성을 "맞추기"보다 6-100 kPa로 "변형"한 이유로 읽을 수 있습니다.
2. **공개 twin은 물리적 잡기를 우회합니다.** 마찰만으로는 세 take 모두 폼이 손에서 69-226 mm 미끄러졌고, 닦인 비율도 0.69-0.76에 그쳤습니다. 30회를 더 시도해도 미끄럼을 10 mm 아래로 줄이지 못했습니다(`04` C4). 같은 저장소의 에이전트 작업 규칙은 latch·weld·attachment를 금지합니다(`examples/live/AGENTS.md`, 2026-10-22 추가). 즉 논문 시점의 데모가 이후 규칙을 어깁니다.
3. **강성 변형 범위가 실제 잡기 가능 범위와 겹치지 않습니다.** Fig. 7e의 6·20 kPa는 공개 twin의 집게 조건에서 스펀지가 패드 사이로 흘러나가는 범위입니다(`04` C6). GaP가 이 범위에서 어떻게 닦았는지는 공개 자료로 알 수 없습니다.
4. **에이전트의 기여와 사람의 기여가 분리되지 않았습니다.** "human-in-the-loop"를 강조하지만, 56분 동안 사람이 몇 번 개입했는지 기록이 없습니다. 저장소 README의 팁("가끔 물리와 기하를 재구성하라고 상기시켜라")은 개입이 필요했다는 뜻입니다.
5. **재현에 필요한 자산 상당수가 비공개입니다.** 18개 과제의 정의(테스트 코드가 `vabar` 컬렉션이라고 부르는 것), UR5e, 실제 로봇 실행 코드, GaP 닦기 그래프가 없습니다. 논문 시점에는 CPU 실행도 지원되지 않았습니다(2026-10-18 추가).

03. 수치 재계산과 toy 해석

스크립트는 `analysis/table_check.py`(결과 `table_check_results.json`)와 `analysis/identifiability_toy.py`(결과 `identifiability_toy_results.json`)입니다. 모두 numpy·scipy로 단일 스레드에서 1초 안에 끝납니다. 공식 데모 수치는 저장소 `examples/08_demos/wipe_the_bar/`(커밋 `81f57c9`)의 README와 `bar.py`에서 옮겼습니다.

1. Preston 마모: 공개 twin의 마커 줄 지우기 (C5)

공개 twin은 마커 줄을 2.5 mm 셀로 나누고, 각 셀의 잉크를 Preston 법칙으로 줄입니다.

```
d(ink)/dt = − k_p · p · v      (k_p = 1.2×10⁻² 1/(Pa·m), p < 300 Pa이면 마모 없음, ink ≤ 0.1이면 "clean")
```

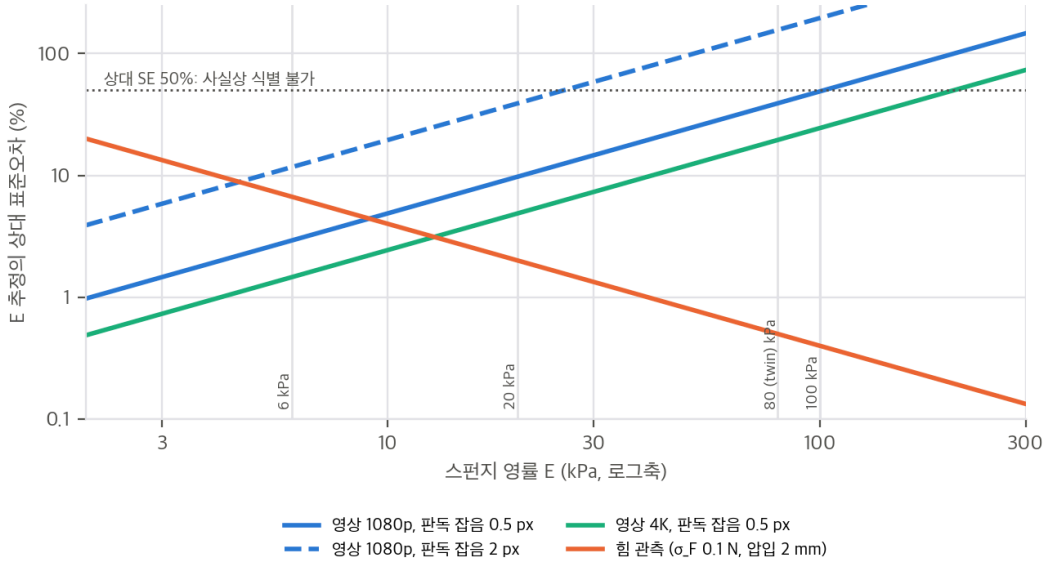
- **주석 검산:** `bar.py`는 "4 N, 30 mm/s 스트로크면 셀 하나를 약 2초에 지운다"고 적었습니다. 4 N을 스펀지 면(50 × 75 mm)에 고르게 나누면 평균 압력이 1,067 Pa이고, 잉크 0.9를 지우는 데 0.9 / (k_p · p · v) = **2.34 s**가 걸립니다. 주석과 맞습니다.
- **한 스트로크의 마모는 속도와 무관합니다.** 스펀지 긴 변 L(75 mm)이 셀을 지나는 시간은 L / v이므로, 한 번 지날 때 마모량은 k_p · p · L입니다. 평균 압력이면 **0.96**이라 한 스트로크로 거의 지워집니다. 측정된 첫 스트로크 0.91과 같은 자릿수입니다.
- **최대 압력은 평균과 두 자릿수 차이가 납니다.** 측정 최대 압력은 84 kPa로 평균의 약 80배이고, 이 압력이면 셀 하나가 0.03초에 지워집니다. "닦인 비율"은 압력이 몰린 곳과 300 Pa 문턱 아래 띠의 비율에 따라 정해지는 지표입니다.
- **함의:** 네 번 닦아도 0.955-0.959에 머무는 것은 문턱 아래 띠와 접촉이 끊기는 셀 때문으로 보입니다(가설). 논문의 "닦인 면적 100%"는 GaP가 만든 다른 rollout의 값이라 직접 비교할 수 없습니다. 다만 같은 법칙과 문턱을 쓴다면 100%는 문턱 아래 셀이 하나도 없어야 나오는 값입니다.

2. 영상만으로 강성을 맞출 수 있는가 (식별성 toy)

논문은 사람이나 에이전트가 영상과 시뮬레이션을 **눈으로** 비교해 물성 값을 고친다고 씁니다(Sec. 4.1, 4.3). 가장 단순한 모델로 그 비교의 해상도를 계산했습니다.

- **모델:** 스펀지 면을 4 N으로 누를 때 선형 압축 δ = F h / (E A)입니다(A = 50 × 75 mm, h = 30 mm).
- **시각 관측:** 화면 폭이 0.6 m인 영상에서 δ를 픽셀로 읽습니다. 판독 잡음은 0.5 px와 2 px 두 경우입니다.
- **힘 관측:** 알려진 2 mm 압입에서 힘을 0.1 N 잡음으로 읽습니다.
- **표준오차:** 해석식은 SE_vision(E) = σ_δ · E² · A / (F h), SE_force = σ_F · h / (A · δ_c)이고, Monte Carlo로 확인했습니다. 20 kPa에서 해석 1.95 kPa, Monte Carlo 1.96 kPa입니다.

E	압축 δ	1080p에서	4K에서
6 kPa	5.33 mm	17.1 px	34.1 px
20 kPa	1.60 mm	5.1 px	10.2 px
80 kPa (공개 twin)	0.40 mm	1.3 px	2.6 px
100 kPa	0.32 mm	1.0 px	2.0 px



toy: 50×75×30 mm 스펀지를 4 N으로 누를 때 선형 압축 δ = F h /(E A), 화면 폭 0.6 m 가정. analysis/identifiability_toy.py

- **영상으로는 E가 약 100 kPa를 넘으면 사실상 정해지지 않습니다.** 상대 SE가 50%를 넘는 경계는 1080p·0.5 px에서 104 kPa, 1080p·2 px에서 26 kPa, 4K·0.5 px에서 206 kPa입니다.
- **영상에서 구분되는 차이와 구분되지 않는 차이가 있습니다.** Fig. 7e의 20 kPa와 100 kPa는 1080p·0.5 px에서 5.8σ 차이라 구분됩니다. 반면 공개 twin의 80 kPa와 100 kPa는 0.36σ 차이라 영상으로는 같습니다.
- **힘 관측은 오차가 E와 무관하게 일정합니다(0.4 kPa).** 그래서 단단한 물체일수록 힘 관측이 유리하고, 무른 물체(10 kPa 미만)에서는 오히려 영상이 낫습니다.
- **해석:** 논문이 힘 피드백을 따로 둔 동기(Sec. 4.2 "관찰만으로는 접촉을 판단하기 어렵다")와 강성을 맞추는 대신 넓게 변형한 선택(Fig. 7e)을 정량적으로 뒷받침합니다. 반대로, 에이전트가 "영상과 맞는 시뮬레이션"을 만들었다는 주장은 기하와 운동에는 강하고 물성에는 약하다는 뜻입니다.
- **한계:** 선형 일축 압축, 고정 시야, 판독 잡음은 모두 가정입니다. 실제 영상의 원근, 가림, 흐림은 이보다 나쁩니다.

3. 강성과 잡기: 공개 twin이 80 kPa를 쓴 이유 (C6)

`bar.py`의 FOAM 주석에 따르면, 80 kPa 폼은 패드당 8.6-8.9 N으로 50 mm에서 45.5 mm로 눌립니다(변형률 9%). 이를 선형 탄성으로 보면 유효 패드 면적은 약 12 cm²입니다. 같은 조건에서 각 강성의 예상 변형률은 다음과 같습니다.

E	예상 변형률	저장소의 관찰
6 kPa	120% (불가능, 완전히 눌림)	-
20 kPa (영상 속 부엌 스펀지)	36%	"패드 사이로 흘러나가 집게가 맞닿았다"
80 kPa (공개 twin)	9%	잡히지만 마찰만으로는 닦는 동안 미끄러짐 → latch
100 kPa	7%	-

선형 추정은 20 kPa에서도 잡힐 것처럼 보이지만, 실제 FEM에서는 큰 변형 때문에 폼이 흘러나갔습니다. 어느 쪽이든 **Fig. 7e의 6·20 kPa는 공개 twin의 집게 조건으로 스펀지를 들 수 없는 범위**입니다. GaP가 이 강성 변형에서 어떻게 닦았는지(latch를 썼는지)는 공개 자료로 알 수 없습니다.

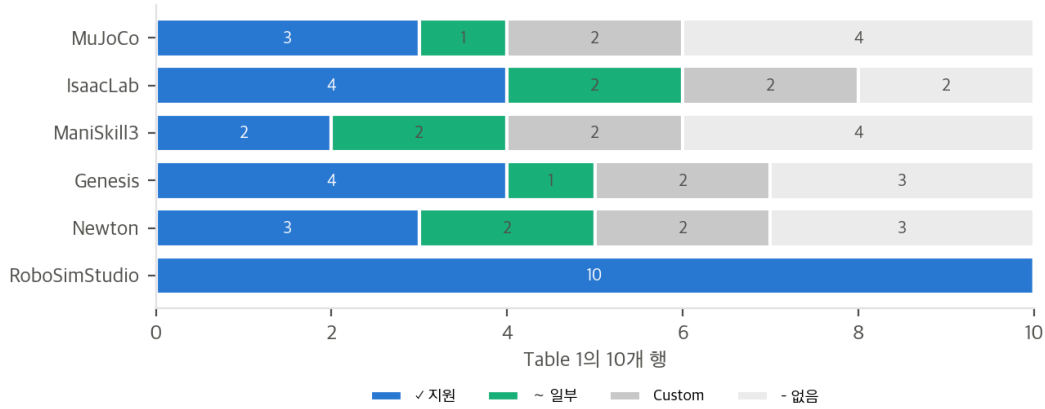
4. GaP 곡선의 분모 (C2)

Fig. 1과 Fig. 7f의 값은 반복 0·9·11·14에서 0·29·71·100%입니다. 이 네 값이 k/n을 정수 %로 반올림해서 나오는 n을 60까지 찾으면 다음과 같습니다.

```
n = 7, 14, 17, 21, 24, 28, 31, 34, 35, 38, 41, 42, 45, 48, 49, 51, 52, 55, 56, 58, 59
```

30은 없습니다. 30개 배치의 성공 비율이라면 29% 대신 26.7%나 30.0%, 71% 대신 70.0%나 73.3%가 나와야 합니다. 따라서 이 곡선은 본문 말대로 닦인 면적 같은 연속량이거나, 30개가 아닌 다른 개수에 대한 비율입니다. Fig. 1의 y축 이름 "completion rate"는 성공률로 읽히기 쉬워서 오해의 여지가 있습니다.

5. Table 1의 구조 (C1)



RoboSimStudio만 ✓인 행 5개는 모두 이 논문이 정의한 워크플로(구조화 피드백, 공동 정제, 영상·힘·실패 기반 Real2Sim). 출처: Table 1.

RSS는 10개 행이 모두 ✓입니다. 다른 다섯 프레임워크는 2-4개입니다. RSS만 ✓인 행은 5개입니다(구조화된 에이전트 피드백, 사람-에이전트 공동 정제, 영상 기반·힘 피드백·실패 기반 Real2Sim). 모두 이 논문이 정의하고 구현한 워크플로입니다. 나머지 물리 기능 5행에서 RSS의 ✓는 대부분 Newton 위에서 물려받은 것입니다. 예를 들어 Newton 자체도 강체·변형체는 ✓이고 결합은 ~입니다.

04. 코드 검증: 논문 ↔ 구현 대조, 수치 재계산, 구현 가능성

- **대상:** `robosimstudio/robosimstudio` @ `81f57c9` (2026-10-28, HEAD, shallow clone). 논문 시점 상태는 첫 커밋 `73739ab` (2026-10-14, v2와 같은 날)의 파일 트리로 확인했습니다. 저장소에는 커밋이 12개뿐이고, 첫 커밋 하나에 대부분이 들어 있습니다.
- **범위:** 코드·문서·테스트 읽기, PyPI 메타데이터 조회(macOS arm64 wheel 여부), CPU 재계산만 했습니다. 설치·실행은 하지 않았습니다.

1. 모듈 매핑

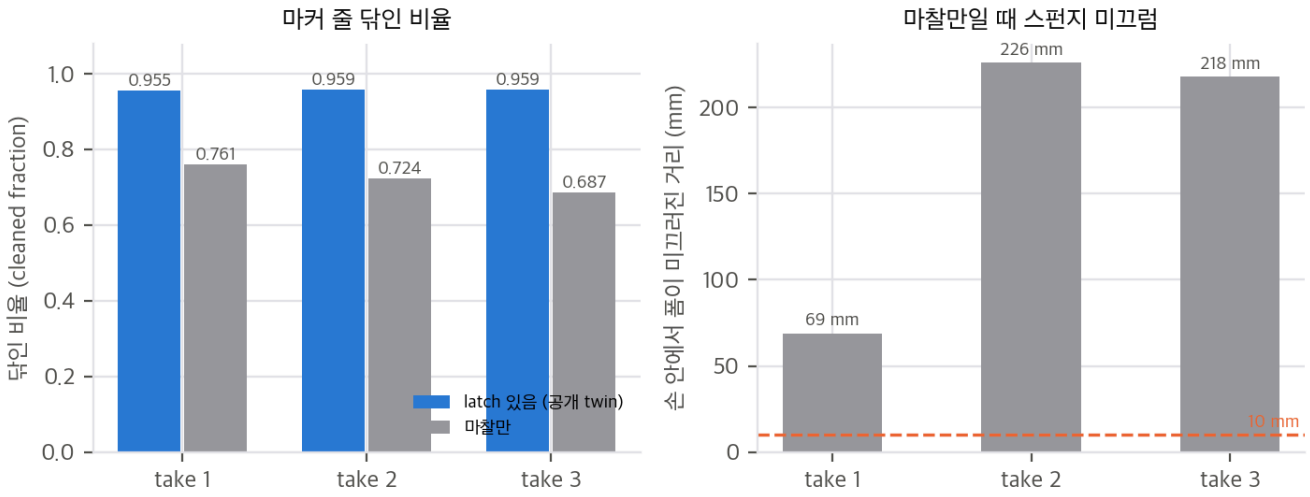
논문	구현 위치	일치 여부
물체는 "무엇인지", 장면은 "어느 solver인지" 선언 (Sec. 3)	`robosimstudio/specs/physics.py` (ArrangementSpec: 영역별 backend, coupler, 충돌 정책), `specs/arrangement.py`	✅ 데이터로 정의된 배치(arrangement)를 이름으로 고릅니다
기본 solver: 관절 MuJoCo-Warp, 천·케이블 VBD, 연성체 외부 엔진	`specs/arrangement.py: GENERIC` — articulated `mujoco`, soft `superdex` (substeps 2), 입자 재료 `mpm`, 그 밖 `vbd`, coupler `proxy`	✅ 논문 이후 MPM(알갱이·액체·반죽)이 추가됐습니다
외부 엔진과 자세·접촉 wrench 교환	`engines/cosim.py` (CosimRig: 자세 읽기 → 외부 엔진 substep → 반력 수확 → Newton에 기록), `engines/feedback.py` (WrenchAccumulator)	✅
인스펙터: 단위·범위, 실행 중 변경 vs 재빌드 표시	`specs/physics.py` 의 ParamSchema("단위, 범위, 변경 비용 tier"), `robosimstudio_studio/inspector.py`, `tuning/`	✅
에이전트 반복 인터페이스	`robosimstudio_real2sim_agent/`: 저장할 때마다 `scene.py`를 다시 빌드하는 live 페이지 + Unix socket CLI(`./ctl`), `compare.py` (사진 대비 크기·위치 수치), `checks.py` (take ↔ 영상 timeline, gate: 빌드·유한성·카메라·가라앉음·표류), `frames.py` (영상에서 시각이 찍힌 정지 화면)	⚠️ D5
힘 피드백 원격 조종: 속도 의존 deadband, 멈춤·지연 시 해제, 측정 열림 사용	`robots/teleop/force_feedback.py` (`deadband`, `velocity_deadband`, stale 샘플이면 disarm, "measured jaw" 사용), 리더별 100 Hz 스레드	✅ 논문 서술과 일치합니다. 리더 하드웨어는 D7
로봇: Panda, UR5e, 단팔·양팔 YAM	`robots/embodiments/`: panda, yam_bimanual, yam_i2rt 등	⚠️ D6 (UR5e 없음)
IK PyRoKi, 계획 cuRobo	`pyproject.toml`: pyroki를 `95afccc`에 고정, cuRobo는 `curobo` extra(cu12)	✅
privileged 상태와 관측 분리	`robosimstudio_policy_agent/sim_tools.py` ("in-process 모드에서만" 등록), `examples/07_policies/02_without_privilege.py`	✅ 구조적으로 분리되어 있습니다
GaP 정책 개선	`robosimstudio_policy_agent/` (GaP 런타임 facade, 도구), GaP 런타임 `KeplerC/graph-as-policy@09e45f8`	⚠️ D2 (닦기 그래프 없음)
스펀지 닦기 twin, Preston 마모	`examples/08_demos/wipe_the_bar/` (`bar.py`, `routine.py`), `engines/superdex/bridge.py` (접촉 patch별 압력)	⚠️ D3, D4

논문	구현 위치	일치 여부
18개 과제 (Fig. 5)	공개 저장소에는 데모 3개(wipe, cable, towel)와 재료별 채점 예제 11개	❌ D1

2. 불일치와 논문에 적히지 않은 구현 요소

#	내용	영향
D1	**Fig. 5의 18개 과제 정의가 공개 저장소에 없습니다.** 과제는 `tasks/` 디렉터리("root")에서 읽는데, 저장소에 그 디렉터리가 없습니다. 테스트는 `vabar/assets/vab_tool_sort`를 "컬렉션 없이 설치하면 없음"이라고 부릅니다(`tests/test_panda_vendored.py`). 공개된 것은 양팔 YAM 데모 3개(`08_demos`)와 재료별 채점 예제(`06_tasks`)입니다. 프로젝트 페이지도 "More Real2Sim Examples Coming Soon"이라고 씁니다.	18개 과제를 다시 만들 수 없습니다. Sec. 4.1의 결과는 확인할 수 없습니다.
D2	**GaP 닦기 정책, 30개 배치, 강성 변형 스윕, 실제 로봇 실행 코드가 없습니다.** 공개된 policy 예제는 pick-and-place 그래프뿐입니다.	Fig. 7e-h를 재현할 수 없습니다.
D3	**공개 twin은 스펀지를 손에 고정(latch)합니다.** 잡기가 안정되면 패드 근처 폼 노드 106개를 손에 고정하고 **Open**에서 풉니다. 마찰만으로는 세 take에서 폼이 69·226·218 mm 미끄러졌고, 닦인 비율은 0.761·0.724·0.687였습니다. 30회 더 바꿔 시도했지만(마찰, 그립 12 N, 누름 1.5-4 N, 기울기, 속도, 볼록 분해) 미끄럼을 10 mm 아래로 줄인 경우는 없었습니다(README "Why a latch"). 막대도 블록에 용접(weld)되어 있습니다.	논문은 latch를 언급하지 않습니다. 같은 저장소의 에이전트 작업 규칙(`examples/live/AGENTS.md`, 2026-10-22 추가)은 "latch, weld, attachment 금지, 손가락 접촉으로만 움직일 것"을 완성 조건으로 둡니다. 논문 시점 데모가 이 규칙을 지키지 않습니다.
D4	**공개 twin의 스펀지는 영상 속 스펀지보다 4배 단단합니다.** `bar.py`는 "영상의 부드러운 부엌 폼(20 kPa)은 8 N 집게에서 패드 사이로 흘러나가 들리지 않아" 80 kPa 폼으로 바꿨다고 적었습니다.	영상과 "맞춘" twin이 물성에서는 영상과 다릅니다. Fig. 7e의 6·20 kPa 변형은 이 집게 조건에서 잡을 수 없는 범위입니다(`03` §3).
D5	**에이전트용 "시공간 피드백"의 실체는 시각 비교 수치와 gate입니다.** `compare.py`는 사진 대비 크기 비율과 픽셀 위치 차이를, `checks.py`는 take의 물체 이동과 영상 timeline 대비, 빌드·유한성·가라앉음·표류 검사를 줍니다. 논문이 나열한 "접촉 사건, 잡기·미끄러짐" 텍스트 피드백을 만드는 전용 모듈은 찾지 못했습니다.	에이전트가 접촉 문제를 풀려면 `./ctl eval`로 시뮬레이터 상태를 직접 읽어야 합니다(Python 실행). 기능은 있지만 논문 서술보다 원시적입니다.
D6	**UR5e가 공개 저장소에 없습니다.** 레지스트리는 "콘텐츠로 배포되는 로봇은 벤치마크 계층을 import해야 등록된다"고 안내합니다. 공개된 것은 Panda, YAM 계열, Robotiq 2F-85 그리퍼입니다.	Fig. 5에서 UR 계열 팔이 보이는 과제는 공개 자산으로 만들 수 없습니다.
D7	**힘 피드백 리더 암은 GELLO가 아니라 YAM 리더입니다.** Feetech STS3215 서보, CH340 USB 어댑터, `lerobot` 버스 드라이버, udev 규칙과 보정 파일을 씁니다. 공개 예제의 힘 피드백 장면은 패널과 상자이고, 공구 걸기(Fig. 6)가 아닙니다.	논문의 GELLO 인용은 개념 출처로 읽어야 합니다. 하드웨어 목록은 저장소 README가 정확합니다.
D8	**CPU 실행은 논문 이후에 추가됐습니다**(2026-10-18, 커밋 `a2fe723`). CPU에서는 MuJoCo C가 팔·강체를, SuperDex가 천·가방·로프·연성체를 맡습니다. 종이·테이프·붓·MPM·물은 거부됩니다. 로프는 단방향(케이블이 닿은 것을 밀지 않음)이고, SDF·hydroelastic 접촉과 다중 world도 쓸 수 없습니다(`examples/install.md`).	논문의 VBD 기반 장면(케이블, 필름, 수건, 열전사)은 CPU에서 다른 물리로 대체되거나 거부됩니다.
D9	**렌더러가 플랫폼에 따라 다릅니다.** 경로 추적 렌더러 OVRTX(1.8 GB)는 RT core가 있는 NVIDIA GPU와 Linux·Windows에서만 씁니다. 그 밖에는 Newton의 tiled 렌더러가 **collider만** 그립니다.	에이전트가 사진과 렌더를 비교하는 루프(`compare_still`)는 RT GPU가 없으면 질감 없는 collider 그림으로 비교하게 됩니다. A100·H100·B200도 RT core가 없어 같습니다.

#	내용	영향
D10	공식 테스트 기록(README "Measured")은 RTX PRO 6000 Blackwell에서 했습니다. 닦기 데모 README는 "superdex extra와 GPU가 필요하다"고 쓰지만, 설치 문서는 SuperDex 스펀지가 CPU에서 돈다고 씁니다.	문서끼리 다릅니다. 이 맥에서 닦기 twin이 도는지는 실행해 봐야 알 수 있습니다.
D11	논문 표지(v2)의 소속 번호는 NVIDIA 1, Berkeley 2이고, 프로젝트 페이지는 Berkeley 1, NVIDIA 2입니다. arXiv 제목에는 "for Human and Agent"가 있고 PDF 제목에는 없습니다.	사소한 표기 차이입니다.



출처: 공식 저장소 examples/08_demos/wipe_the_bar/README.md (커밋 81f57c9). 마찰만으로 30회 더 시도해도 미끄럼 10 mm 미만인 경우 없음.

우리 분석: 공식 wipe_the_bar 데모의 latch 유무별 닦인 비율과 미끄럼(저장소 README 수치).

3. 수치 재계산 (`analysis/table_check.py`, `analysis/identifiability_toy.py`)

#	대상 (출처)	재계산	판정
C1	Table 1	RSS 10/10 ✓, 다른 프레임워크 ✓ 2-4개. RSS만 ✓인 5행은 모두 이 논문의 워크플로	기능 목록에 가까움
C2	GaP 곡선 0·29·71·100% (Fig. 1, 7f)	k/n 반올림으로 나오는 n은 7, 14, 17, 21, 24 …이고 **30은 아님**	30개 배치의 성공률이 아님. Fig. 1 y축 이름 "completion rate"는 오해 소지
C3	에이전트 구축 시간 (Fig. 7c)	첫 닦기까지 42분(전체 56분의 75%). 56분은 한 에이전트만 보고됨	두 번째 에이전트 시간 없음
C4	latch 효과 (저장소 README)	latch 0.958 vs 마찰만 0.724, 차이 23.4%p, Welch p = 0.008(n = 3)	공개 twin의 닦기 성능은 latch가 만듦
C5	Preston 주석 "4 N, 30 mm/s 면 셀 하나에 약 2초" (`bar.py`)	면 평균 압력 1,067 Pa → 2.34 s. 한 스트로크 마모 = k_p·p·L = 0.96(속도 무관)	✅ 주석과 측정 첫 스트로크 (0.91)와 맞음
C6	강성과 잡기 (`bar.py` FOAM 주석)	80 kPa에서 변형률 9% → 유효 패드 면적 12 cm². 선형 예측 6 kPa 120%, 20 kPa 36%	Fig. 7e 범위의 절반은 잡을 수 없는 강성
C7	v1 ↔ v2	13쪽 → 14쪽, 참고문헌 51 → 58개, 단어열 유사도 0.87. Fig. 1 라벨, 관련 연구(MAESTRO, ASPIRE, ENPIRE, DexAgent, RPG, Push-T), Real2Sim 문단(frontier 모델의 시각 재구성 한계) 추가	방법·결과 수치는 그대로
V1	영상만으로 E 식별 (toy)	1080p·0.5 px에서 E > 104 kPa면 상대 SE > 50%. 80 vs 100 kPa는 0.36σ. 힘 관측 SE는 0.4 kPa로 일정	`03` §2

4. 구현 가능성 판단

4.1 공식 코드를 그대로 쓰는 경우

항목	요구 사항	비고
OS / Python	Linux x86-64 + Python 3.12 (개발·테스트 플랫폼). "macOS와 Windows는 테스트하지 않음"	이 맥에는 Python 3.12.7이 있고 `uv`는 없습니다
GPU	선택. NVIDIA GPU가 있으면 Warp CUDA, MuJoCo-Warp, VBD, MPM, CUDA graph를 씁니다. 없으면 CPU 경로(D8)	경로 추적 렌더러는 RT core GPU + Linux·Windows만
주요 의존성	newton[sim] 1.7, warp 1.18, mujoco 3.15, viser, coacd 1.0.11, pyroki(git, jax), imageio[ffmpeg]. 선택: superdex(cpu), genesis(+torch), curobo(cu12), gap(torch, cuRobo, transformers), lerobot-feetech	PyPI 조회 결과 warp·coacd·superdex는 macOS arm64 wheel이 있고, newton·mujoco-warp·viser는 순수 Python입니다
용량	저장소 checkout 약 210 MB(로봇 자산 포함). rtx 그룹 1.8 GB(Linux·Windows만)	
코딩 에이전트	Claude Code나 Codex(유료 구독 또는 API). 영상·사진이 공급자 서버로 갑니다	논문의 "약 1시간"은 이 비용과 사람 감독 시간을 포함하지 않습니다
하드웨어 (선택)	힘 피드백: Feetech STS3215 YAM 리더 암 + CH340 어댑터. 실제 로봇: 양팔 YAM	

판단: 도구 자체는 **설치해서 쓸 수 있는 수준으로 공개**되어 있습니다(Apache-2.0, 설치 문서, 예제, 테스트). 논문의 결과(18개 과제, GaP 닦기, 실제 로봇)는 핵심 자산이 비공개라 **재현할 수 없습니다**(D1, D2, D6).

4.2 이 맥(Apple M4 Pro, CUDA 없음)에서

공식 문서의 CPU 매트릭스(`examples/install.md`)와 Fig. 3의 solver 표시를 합쳐 18개 과제를 추정했습니다. 과제 정의가 비공개라 재료 종류는 이름에서 짐작한 것입니다.

과제 (Fig. 5)	추정 재료	이 맥(CPU)에서
공구 정리, 주사기 폐기, 바구니 담기, 공구 걸기, 접시 꽂기, 서랍 열기, 상자 뒤집기	강체·관절	가능(MuJoCo C). 단 UR5e 과제는 자산 없음
기어 맞물림, 커넥터 삽입	강체 + 정밀 메쉬 접촉	어려움: CPU에는 SDF·hydroelastic 접촉이 없음
케이블 배선, 도관 통과	로프(논문은 VBD)	다른 물리로 가능: SuperDex 로프, 단방향
수건 접기, 잎 누르기	천·쉘(논문은 VBD)	다른 물리로 가능: SuperDex 천
필름 벗기기, 박스 테이프 자르기	테이프·절단(VBD)	테이프는 거부, 절단은 느리게 가능(1-2 s/step)
스펀지 닦기	FEM(SuperDex)	가능해 보임(macOS arm64 wheel 있음). 데모 README는 GPU 필요라고 씀(D10)
띠 연마, 열전사	접촉 + 표면 법칙, 필름(VBD)	띠 연마는 가능해 보임, 열전사는 불확실

- **할 수 있는 것:** `examples/`의 강체·관절 장면, Viser studio, `./ctl` 기반 에이전트 루프(Unix socket은 macOS에서 동작), CPU 데모 기록. 설치에는 승인이 필요하고, 실제 동작은 확인하지 않았습니다.
- **할 수 없는 것:** CUDA 경로 전체(MuJoCo-Warp, VBD, MPM, cuRobo 계획), 경로 추적 렌더러, 다중 world, GaP 런타임(torch + cuRobo, cuRobo가 CUDA 전용이라 불가로 추정). Apple GPU(MPS)는 이 스택이 쓰지 않습니다. 저장소가 고르는 장치는 `cpu`와 `cuda` 뿐입니다(`robosimstudio/machine.py`).

- **속도:** 공식 문서 기준 CPU 한 스레드에서 VBD 수건(1,922 입자)은 실시간의 0.003배, 절단은 GPU 7.6 ms 대비 step당 1-2 s입니다. 맥에서는 "보고 만지는" 용도로는 쓸 수 있지만, 데이터 생성이나 정책 개선 루프에는 비현실적입니다.

4.3 아이디어만 가져와 재구현하는 경우

구성 요소	난이도	메모
"물성·solver를 편집 가능한 항목으로" 장면 기술	중간	이미 쓰는 시뮬레이터(MuJoCo, Isaac Lab) 위에 인스펙터와 파라미터 schema(단위·범위·재빌드 여부)를 얹는 일
에이전트 루프(저장하면 다시 빌드, 사진 비교 수치, gate)	낮음-중간	`robosimstudio_real2sim_agent` 의 구조(live 페이지, socket, compare, gate, log.jsonl)가 짧고 독립적이라 옮기기 쉽습니다
힘 피드백 원격 조종	중간	리더 암 하드웨어가 필요합니다. deadband·stale 처리 같은 안전 로직은 `force_feedback.py` 에서 가져올 수 있습니다
다중 물리 co-sim(강체 + FEM)	높음	Newton과 SuperDex를 직접 쓰는 것이 현실적입니다. 직접 결합을 짜는 것은 권하지 않습니다

요약: RSS는 논문이라기보다 **바로 쓸 수 있는 오픈소스 도구**입니다. 다만 논문의 성과를 판단하려면 비공개 자산이 필요합니다. 이 맥에서는 강체 장면과 에이전트 루프를 시험해 볼 수 있지만, 변형체와 정책 개선은 NVIDIA GPU 서버가 필요합니다.

02. 선행연구와 계보

번호 [n]은 논문(v2)의 참고문헌 번호입니다. GitHub 정보는 2026-10-29에 `gh api`로 확인했고, 공식 저장소는 코드를 직접 읽었습니다. 나머지 논문은 따로 열어 검증하지 않았습니다.

1. 계보도

```
[시뮬레이터]  MuJoCo [9] → Isaac Gym / Isaac Lab [10–12], ManiSkill3 [13], MuJoCo Playground [14], Genesis [6]
              Warp [16] → Newton [3] (MuJoCo-Warp [17], VBD [18], XPBD [19] 통합) · SuperDex [4] (암시적 FEM, 외부 엔진)
                                          |
[Real2Sim]   digital twin / cousin: RialTo [44], URDFormer [45], Digital Cousins [46], Articulate-Anything [47],
             SIMPLER [48], SplatSim [49]  ·  생성형: GenSim [31, 32], RoboGen [50], Holodeck [51], Video2Policy [33]
                                          |
[에이전트 로보틱스]  SayCan [20], Code as Policies [21], VoxPoser [23] → Eureka [27], DrEureka [30]
             → 범용 코딩 에이전트(ReAct [7], Claude Code [1], Codex [2]) → CaP-X [36], GaP [5], ASPIRE [38], ENPIRE [39]
                                          |
                    ★ RoboSimStudio: 시뮬레이터의 모델 선택을 "편집 가능한 항목"으로 노출해 사람·에이전트 공동 Real2Sim
```

차별점: 기존 Real2Sim 연구는 대부분 재구성이나 생성 파이프라인 하나를 제안합니다. RSS는 파이프라인이 아니라 **작업대**입니다. solver·접촉 모델·물성을 바꾸고 결과를 보는 루프를 사람과 코딩 에이전트에게 함께 제공하는 것이 핵심입니다. 같은 그룹(Goldberg lab)의 GaP [5], CaP-X [36], Push-T 에이전트 연구 [42]와 이어지는 "코딩 에이전트가 로봇 문제를 직접 실험하며 푼다"는 흐름의 시뮬레이션 쪽 기반이기도 합니다.

2. 구성 요소별 출처

역할	출처	이 논문에서의 쓰임
물리 백본	Newton [3] (Warp [16] 위)	장면 조립, 결합, 렌더링
관절·강체 solver	MuJoCo-Warp [17] (CPU에서는 MuJoCo C)	로봇 팔, 강체
천·케이블 solver	VBD [18]	기본 "그 밖의 영역" solver
연성체 FEM	SuperDex [4] (`facebookresearch/project_superdex`)	스펀지, 가방. CPU에서는 천·로프도 담당
두 번째 외부 엔진	Genesis [6]	IPC 천·연성체, SPH 물(선택)
브라우저 화면	Viser [8]	studio, 인스펙터, 기즈모
IK / 계획	PyRoKi [55] / cuRobo [56]	팔 IK(기본), 충돌 회피 계획 (`motion="plan"`, CUDA)
정책 개선	Graph-as-Policy [5] (`KeplerC/graph-as-policy`)	스펀지 닦기 정책을 시뮬레이션 피드백으로 개선

역할	출처	이 논문에서의 쓰임
리더 암	GELLO [58] (논문 인용) · 저장소는 Feetech STS3215 서보의 YAM 리더 + `lerobot` 버스 드라이버	힘 피드백 원격 조종
마모 모델	Preston [57]	띠 연마, 마커 줄 지우기
에이전트 루프	ReAct [7], SWE-agent [43]	"고치고-실행하고-관찰" 루프의 근거

3. 이 저장소의 다른 논문과의 관계

논문	관계
Do as I Do (`do_as_i_do/`)	둘 다 "일상 영상 → 물리 시뮬레이션"입니다. Do as I Do는 영상에서 손·물체 4D 궤적과 SAM 3D 메쉬를 자동으로 복원한 뒤 MuJoCo Warp sampling으로 리타겟팅합니다. RSS는 장면 자체를 코딩 에이전트가 손으로 짓습니다. Do as I Do가 다루지 못한 **변형 물체와 접촉 모델 선택**을 RSS가 채우고, RSS가 수작업으로 하는 **기하 복원**을 Do as I Do가 자동화합니다. 서로 보완하는 관계입니다.
FlashDexRetarget (`flashdexretarget/`)	FlashDexRetarget은 Isaac Sim 강체 장면에서 리타겟팅합니다. 둘 다 **물리적이지 않은 보조**를 씁니다. FlashDexRetarget은 물체를 레퍼런스로 끄는 PD 힘을 학습 중에만 쓰고 평가에서는 끕니다. RSS 공개 twin은 스펀지를 손에 고정하는 latch를 과제 내내 씁니다(`04` D3). 보조를 평가에서 빼는지가 결과의 물리적 타당성을 가릅니다.
GraspSense (`graspsense/`)	GraspSense는 SAM 3D 메쉬로 Isaac Lab 장면을 만들고 힘 지도로 파지를 고르지만, 코드상 물체는 강체였습니다. RSS의 SuperDex FEM은 GraspSense가 검증하지 못한 "벽 변형·손상" 주장을 실제 변형체로 확인할 수 있는 도구입니다.
AlphaNIST (`alphanist/`)	AlphaNIST는 Isaac Lab에서 NIST 조립 보드를 학습합니다. RSS의 커넥터 삽입(1 mm 미만 여유)과 같은 접촉 정밀도 문제를 다룹니다. RSS는 CPU에서 SDF·hydroelastic 접촉을 쓸 수 없어, 이런 과제는 GPU가 필요합니다(`04` §4).
Contact-Grounded Policy (`contact_grounded_policy/`)	CGP는 Meta 내부 FEM 시뮬레이터로 visuotactile 정책을 학습했습니다. RSS가 쓰는 SuperDex는 Meta의 공개 암시적 FEM 엔진입니다(`facebookresearch`). 같은 계열인지는 확인하지 못했습니다(가설).

4. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

주제	관련성	근거 수준
시스템 식별(system identification)·베이지안 보정	"영상과 맞을 때까지 값을 고친다"는 문제는 고전적인 물성 식별 문제입니다. 관측 가능성과 식별성 분석 없이 눈으로 맞추면 여러 값이 똑같이 그럴듯해 보입니다(`03` §2).	내 해석
미분 가능한 시뮬레이션으로 물성 추정	같은 Warp·Newton 위에서 경사 기반으로 물성을 맞추는 방법은 에이전트의 시행착오를 줄일 수 있습니다.	내 사전 지식. 서지 확인 필요
도메인 랜덤화 (Tobin et al. 2017 등)	Fig. 7e의 강성·배치 변형은 고전적 도메인 랜덤화입니다. 식별할 수 없는 값을 랜덤화로 처리하는 것은 합리적이지만, 범위가 물리적으로 의미 있어야 합니다(`04` C6).	내 사전 지식

5. GitHub 정리 (2026-10-29 `gh api` 기준)

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인한 커밋
robosimstudio/robosimstudio	이 논문의 도구 전체(라이브러리, studio, 에이전트 페이지, 벤치마크, GaP 연결)	**공식**	Apache-2.0 (로봇 자산은 MIT·BSD)	270	2026-10-28	`81f57c9` (HEAD, clone), `73739ab` (논문 시점 첫 커밋, 트리만)
robosimstudio/robosimstudio.github.io	프로젝트 페이지	공식	표기 없음	0	2026-10-15	`47b0796` (메타데이터만)
newton-physics/newton	물리 백본 (Linux Foundation 프로젝트)	의존성	Apache-2.0	5,861	2026-10-28	`23638bf` (메타데이터만)
NVIDIA/warp	GPU/CPU 커널 언어	의존성	Apache-2.0	7,253	2026-10-29	메타데이터만
google-deepmind/mujoco_warp	MuJoCo GPU판	의존성	Apache-2.0	1,592	2026-10-28	메타데이터만
facebookresearch/project_superdex	암시적 FEM 엔진 (스펀지·가방)	의존성	Apache-2.0	770	2026-10-29	`353abe6` (메타데이터만)
viser-project/viser	브라우저 3D 화면	의존성	Apache-2.0	2,838	2026-09-25	`56712d3` (메타데이터만)
chungmin99/pyroki	IK (jax)	의존성	MIT	1,812	2026-04-01	RSS가 `95afccc`에 고정
NVlabs/curobo	충돌 회피 계획 (CUDA)	선택 의존성	Apache-2.0	1,943	2026-09-10	RSS가 `v0.8.0`에 고정
KeplerC/graph-as-policy	GaP 런타임 [5]	같은 1저자, 선택 의존성	Apache-2.0	1	2026-10-24	RSS가 `09e45f8`에 고정
Genesis-Embodied-AI/genesis-world	두 번째 외부 엔진	선택 의존성	Apache-2.0	30,136	2026-10-28	메타데이터만
wuphilipp/gello_software	GELLO 리더 암 [58]	관련(논문 인용, 저장소는 YAM 리더 사용)	MIT	561	2026-09-14	메타데이터만
i2rt-robotics/i2rt	YAM 로봇 모델 출처	자산 출처	MIT	178	2026-10-28	메타데이터만
KeplerC/microduck-with-rss	사진 한 장 Real2Sim 데모(README 소식)	같은 1저자	Apache-2.0	2	2026-10-26	메타데이터만

한계: 4절의 서지는 내 사전 지식입니다. 블로그나 git에 올리기 전에 원문을 확인해야 합니다.